Lab -1

This script demonstrates a full end-to-end **data preprocessing and cleaning pipeline** on Scikit-Learn's Breast Cancer dataset.
Here is a step-by-step breakdown of what each section does:

1. Dataset Loading & Inducing Missing Values

```
cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df.iloc[10:15, 0] = np.nan
```

- **Loads the dataset:** Scikit-Learn's dataset is converted into a Pandas DataFrame.
- **Injects NaNs:** Rows index 10 through 14 of column 0 (mean radius) are artificially set to np.nan to simulate real-world missing data.

2. Missing Value Imputation

```
df_imputed = df.fillna(df.median())
```

- **Handles missing values:** fillna(df.median()) replaces all NaN values with the column-wise median. Medians are preferred over means here because they are resistant to extreme values.

3. Outlier Detection and Removal (3-Sigma Rule)

```
def remove_distribution_outliers(dataframe, threshold_sigma=3):
    avg, std = dataframe.mean(), dataframe.std()
    clean_mask = (dataframe >= (avg - threshold_sigma * std)) & (dataframe
<= (avg + threshold_sigma * std))
    return dataframe[clean_mask.all(axis=1)]

df_clean = remove_distribution_outliers(df_imputed)
```

- **3σ Filtering:** Under a normal distribution, roughly 99.7% of data falls within 3 standard deviations ($\mu \pm 3\sigma$) of the mean.
- **Row-wise filtering:** .all(axis=1) checks if a row is within the 3σ threshold across **all** numeric columns. If a row has an outlier in even one column, the entire row is dropped.

4. Injecting & Removing Duplicates

```
df_with_dups = pd.concat([df_clean, df_clean.iloc[10:15]])
df_deduplicated = df_with_dups.drop_duplicates().copy()
```

- **Simulates duplicates:** pd.concat manually duplicates 5 rows (indices 10 through 14) onto the end of the DataFrame.
- **Deduplication:** .drop_duplicates() identifies duplicate rows across all features and keeps only the first occurrence.

5. Random Sampling

```
sample_size = 10
seed_value = 42

sample_wo = df_deduplicated.sample(n=sample_size, replace=False,
random_state=seed_value)
sample_w = df_deduplicated.sample(n=sample_size, replace=True,
random_state=seed_value)
```

- **Without Replacement (replace=False):** Draws 10 unique rows. Once a row is picked, it cannot be picked again.
- **With Replacement (replace=True):** Draws 10 rows, allowing individual rows to be sampled multiple times (used in techniques like bootstrapping).
- **random_state=42:** Ensures reproducible results every time the script is executed.

6. Data Discretization (Binning)

```
target_col = df_deduplicated['mean radius']
df_deduplicated['Equal_Width_Bin'] = pd.cut(target_col, bins=4,
labels=["Small", "Medium", "Large", "Max"])
df_deduplicated['Equal_Freq_Bin'] = pd.qcut(target_col, q=4, labels=["Q1",
"Q2", "Q3", "Q4"])
```

- **Equal-Width Binning (pd.cut):** Divides the range of mean radius ($\text{Max} - \text{Min}$) into 4 equal-sized interval widths.
- **Equal-Frequency Binning (pd.qcut):** Divides the data into 4 quantiles such that each bin contains approximately the exact same number of samples.

7. Formatted Output Printing

```
print(tabulate(sample_wo[report_cols], headers='keys',
tablefmt='fancy_grid', showindex=False))
```

- Uses the tabulate library with tablefmt='fancy_grid' to render ascii tables in the terminal for:

1. The sampled rows without replacement.
2. The sampled rows with replacement.
3. The first 10 rows showing the original numerical mean radius alongside its continuous-to-discrete binned representations.

Lab-2

This script performs **Market Basket Analysis (Association Rule Mining)** on movie streaming histories using the mlxtend library. It identifies which movies are frequently watched together and builds rules to predict viewing habits (e.g., *"If a user watches X, they are likely to watch Y"*). Here is the step-by-step breakdown of how the code works:

1. Data Preparation & One-Hot Encoding

```
streaming_history = [
    ['The Batman', 'Inside Man', 'Superman', 'Invincible'],
    ['Inside Man', 'Superman', 'Invincible'],
    ['The Batman', 'Superman', 'Akira', 'Invincible'],
    ['The Batman', 'Inside Man', 'Invincible', 'Superman'],
    ['Inside Man', 'Spiderman', 'Iron Man'],
    ['The Batman', 'Superman', 'Invincible']
]
viewer_encoder = TransactionEncoder()
matrix_array =
viewer_encoder.fit(streaming_history).transform(streaming_history)
movie_df = pd.DataFrame(matrix_array,
columns=viewer_encoder.columns_).astype(bool)
```

- **Raw Transactions:** streaming_history is a list of lists representing 6 different user viewing sessions.
- **TransactionEncoder:** Converts the nested lists into a binary 2D matrix (rows = viewing sessions, columns = movie titles).
- **DataFrame Creation:** Converts the matrix into a boolean Pandas DataFrame where True indicates a user watched that movie, and False indicates they did not.

2. Output Boolean Matrix Display

```
movie_display = movie_df.map(lambda status: 'True' if status else 'False')
print(tabulate(movie_display.head(5), headers='keys',
tablefmt='fancy_grid', showindex=False))
```

- Converts boolean values to 'True'/'False' strings and uses tabulate to print the one-hot encoded matrix cleanly in the terminal.

3. Finding Frequent Itemsets (Apriori Algorithm)

```
MOVIE_SUPPORT_LIMIT = 0.4
frequent_movies = apriori(movie_df, min_support=MOVIE_SUPPORT_LIMIT,
use_colnames=True)
```

- **Support Threshold:** Sets min_support = 0.4 (40%). An item or combination of items must appear in at least 40% of all viewing sessions (at least 3 out of 6 sessions) to be considered "frequent".
- **apriori():** Finds all single movies and groups of movies that meet this minimum support limit.

4. Generating Association Rules

```
MINING_METRIC = 'confidence'
METRIC_THRESHOLD = 0.6
movie_rules_df = association_rules(frequent_movies, metric=MINING_METRIC,
min_threshold=METRIC_THRESHOLD)
```

- **association_rules():** Generates **If-Then** relationships ($A \rightarrow B$, where A is the antecedent and B is the consequent).
- **Filter Criteria:** Keeps only rules with a **confidence** ≥ 0.6 (60%). Confidence measures how often B is watched when A is watched.

Key Metrics Calculated:

- **Support:** The proportion of total transactions that contain both A and B.
- **Confidence:** $P(B\vert{}A) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$ — the conditional probability that a user watches B given they watched A.
- **Lift:** $\frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$ — measures how much more likely B is watched when A is watched compared to B's baseline popularity:
  - $\text{Lift} > 1$: Strong positive relationship.
  - $\text{Lift} = 1$: Independent items.
  - $\text{Lift} < 1$: Negative relationship.

5. Displaying Results

```
rules_display['antecedents'] = rules_display['antecedents'].apply(lambda
group: ', '.join(list(group)))
rules_display['consequents'] = rules_display['consequents'].apply(lambda
group: ', '.join(list(group)))
print(tabulate(rules_display, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

- Converts Python frozenset objects (which mlxtend uses internally for rules) into readable, comma-separated string labels for formatted printing.

Lab-3

This script performs **Market Basket Analysis** on movie watchlists, but with one key difference from the previous code: it uses the **FP-Growth (Frequent Pattern Growth)** algorithm instead of Apriori to extract frequent itemsets.
Here is the step-by-step breakdown:

1. Data Transformation (Transaction Encoding)

```
watchlist_data = [...]
data_encoder = TransactionEncoder()
matrix_fit = data_encoder.fit(watchlist_data).transform(watchlist_data)
encoded_df = pd.DataFrame(matrix_fit,
columns=data_encoder.columns_).astype(bool)
```

- **Raw Data:** Contains 10 viewing sessions (watchlist_data) across various movies.
- **One-Hot Encoding:** TransactionEncoder converts the lists into a boolean matrix ($10 \text{ rows} \times N \text{ unique movies}$).
- **Boolean Conversion:** Explicitly cast to .astype(bool) to prevent deprecation warnings in newer versions of mlxtend.

2. Frequent Pattern Mining via FP-Growth

```
SUPPORT_FLOOR = 0.4
mined_patterns = fpgrowth(encoded_df, min_support=SUPPORT_FLOOR,
use_colnames=True)
```

- **Algorithm Choice (fpgrowth):** Unlike Apriori (which generates candidate itemsets step-by-step and scans the dataset repeatedly), FP-Growth builds a compact **FP-Tree** structure in memory. It finds frequent itemsets without candidate generation, making it significantly faster on large datasets.
- **Support Threshold:** min_support = 0.4 means a movie combination must appear in at least **40%** (4 out of 10) of watchlists to be considered frequent.

3. Association Rule Generation

```
EVAL_METRIC = 'confidence'
CONFIDENCE_FLOOR = 0.6
rule_matrix = association_rules(mined_patterns, metric=EVAL_METRIC,
min_threshold=CONFIDENCE_FLOOR)
```

- Generates directional rules ($A \rightarrow B$) from the frequent itemsets mined by

FP-Growth.

- **Confidence Floor:** Keeps rules where $\text{Confidence} \ge 0.6$ (60% chance that watching movie set A leads to watching movie set B).

Metrics Displayed:

- **support:** Percentage of total sessions containing both A and B.
- **confidence:** Conditional probability $P(B\vert{}A)$.
- **lift:** Strength of the rule relative to random chance (>1 indicates a positive association).

4. Output Formatting

```
rules_report['antecedents'] = rules_report['antecedents'].apply(lambda
items: ', '.join(list(items)))
rules_report['consequents'] = rules_report['consequents'].apply(lambda
items: ', '.join(list(items)))
print(tabulate(rules_report, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

- Unpacks the frozenset objects returned by mlxtend into clean, comma-separated strings for terminal formatting via tabulate.

Key Takeaway: Apriori vs. FP-Growth in these scripts

Feature	Script 2 (Apriori)	Script 3 (FP-Growth)
Method	Candidate generation and testing	Tree-structure memory graph (FP-Tree)
Speed/Scale	Slower on dense datasets	Faster, more memory-efficient
Output	Exact same rules generated for identical datasets/thresholds	Exact same rules generated for identical datasets/thresholds

Lab-4

This script demonstrates a complete **Supervised Machine Learning pipeline using a Decision Tree Classifier** to classify animals into categories (mammal, bird, fish) based on physical traits. Here is the step-by-step breakdown of how the code works:

1. Dataset Creation & Train-Test Split

```
animal_df = pd.DataFrame(dataset_map)
features = animal_df.drop('class', axis=1)
labels = animal_df['class']

X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.3, random_state=42, stratify=labels
)
```

- **Features (X) & Target (y):** The features are physical attributes (eats_meat, has_scales, is_warmblooded, has_gills, no_of_legs), while the target label is the animal class.
- **30% Test Split:** test_size=0.3 sets aside 30% of the dataset (5 samples) for evaluation and 70% (10 samples) for training.
- **Stratification:** stratify=labels ensures that both the training and test sets maintain the same proportion of mammal, bird, and fish classes as the original dataset.

2. Decision Tree Model Training

```
model_tree = DecisionTreeClassifier(criterion='entropy', random_state=42,
max_depth=5)
model_tree.fit(X_train, y_train)
```

- **Entropy & Information Gain:** criterion='entropy' uses **Information Gain** (calculating reduction in disorder/entropy) to determine the best splits at each node.
- **Hyperparameters:** max_depth=5 limits how deep the tree can grow to prevent overfitting.

3. Tree Visualization & Saving

```
fig, ax = plt.subplots(figsize=(6, 4))
plot_tree(
    model_tree, feature_names=features.columns,
    class_names=model_tree.classes_, filled=True, rounded=True, fontsize=6,
ax=ax
)
```

```
plt.savefig("animal_decision_tree_visualization.png", dpi=300,
bbox_inches='tight')
plt.show()
```

- Renders the generated decision rules (nodes, split criteria, sample counts, and predicted classes).
- Saves the resulting diagram as a high-resolution PNG image (animal_decision_tree_visualization.png).

4. Predicting on Unseen Data

```
unseen_animals = pd.DataFrame({
    'eats_meat': [1, 0, 1, 0],
    'has_scales': [0, 0, 1, 0],
    'is_warmblooded': [1, 1, 0, 1],
    'has_gills': [0, 0, 1, 0],
    'no_of_legs': [4, 2, 0, 4]
})
predictions_summary = pd.DataFrame({
    "Animal No.": [f"A{i+1}" for i in range(4)],
    "Prediction": model_tree.predict(unseen_animals)
})
```

- Passes 4 completely new animal feature vectors through model_tree.predict() to classify them into mammal, bird, or fish.

5. Model Evaluation

```
y_pred = model_tree.predict(X_test)

# Confusion Matrix
cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred, labels=model_tree.classes_),
    index=model_tree.classes_,
    columns=model_tree.classes_
)

# Accuracy & Metrics
accuracy = accuracy_score(y_test, y_pred)
report_dict = classification_report(y_test, y_pred,
```

```
target_names=model_tree.classes_, output_dict=True, zero_division=0)
```

Evaluates performance on the reserved 30% test set (X_test):

- **Confusion Matrix:** Shows actual vs. predicted label counts across classes to spot where misclassifications occur.
- **Accuracy Score:** Percentage of total correct predictions ($\frac{\text{Correct}}{\text{Total}}$).
- **Classification Report:** Breakdown of **Precision**, **Recall**, and **F1-Score** per class:
  - **Precision:** Accuracy of positive predictions.
  - **Recall:** True positive rate (how many actual instances of a class were captured).
  - **F1-Score:** Harmonic mean of Precision and Recall.
  - zero_division=0: Prevents errors if a class has no predicted/true samples in the test split.

Lab-5

This script demonstrates how to implement a **Gaussian Naive Bayes (GaussianNB)** classifier on the classic **Iris flower dataset** using Scikit-Learn.
Here is the step-by-step breakdown of how the code works:

1. Dataset Loading & Train-Test Split

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
```

- **Iris Dataset:** Contains 150 samples measuring 4 physical features (sepal length, sepal width, petal length, petal width) across 3 flower species (setosa, versicolor, virginica).
- **Stratified Split:** test_size=0.3 allocates 30% (45 samples) for testing and 70% (105 samples) for training. stratify=y ensures equal representation of all three flower species in both sets.

2. Model Training with Gaussian Naive Bayes

```
model_nb = GaussianNB()
model_nb.fit(X_train, y_train)
y_pred = model_nb.predict(X_test)
```

- **Bayes' Theorem:** The classifier applies Bayes' Rule to calculate the probability of a sample belonging to a given class:
  $$P(y \mid X) = \frac{P(X \mid y) \cdot P(y)}{P(X)}$$
- **Gaussian Assumption:** Because the continuous feature values in the Iris dataset are real-valued, GaussianNB assumes that continuous values associated with each class follow a **normal (Gaussian) distribution**.
- **Independence Assumption:** It assumes features are conditionally independent given the class label.

3. Sample Prediction Inspection

```
predictions_summary = pd.DataFrame({
    "Sample Index": X_test.index[:5],
    "Actual Class": [iris.target_names[i] for i in y_test[:5]],
    "Predicted Class": [iris.target_names[i] for i in y_pred[:5]]
})
```

```
print(tabulate(predictions_summary, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

- Extracts the first 5 test samples and maps target integers (0, 1, 2) to their human-readable species names (setosa, versicolor, virginica) for a direct visual side-by-side comparison of actual vs. predicted labels.

4. Model Evaluation & Performance Metrics

```
# Confusion Matrix
cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=iris.target_names,
    columns=iris.target_names
)

# Accuracy & Metrics Report
accuracy_score(y_test, y_pred)
report_dict = classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True)
```

Evaluates the model on the 45 test instances:

- **Confusion Matrix:** Displays true classes along rows and predicted classes across columns to highlight misclassified flower species.
- **Accuracy Score:** Percentage of correct overall predictions ($\frac{\text{Correct}}{\text{Total}}$).
- **Classification Report:** Detailed statistical breakdown for each species:
  - **Precision:** Ratio of correctly predicted flowers to the total predicted for that species.
  - **Recall:** Ratio of correctly predicted flowers to all actual flowers of that species in the test set.
  - **F1-Score:** Harmonic mean balancing Precision and Recall.

Lab-6

This script demonstrates a **Support Vector Machine (SVM) Classification pipeline** using Scikit-Learn's LinearSVC model on the **Iris dataset**.
Here is the step-by-step breakdown of how the code works:

1. Dataset Loading & Train-Test Split

```
iris = datasets.load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

- **Dataset:** Uses the Iris dataset containing 150 instances with 4 continuous features (sepal length, sepal width, petal length, petal width).
- **20% Test Split:** test_size=0.2 sets aside 20% (30 samples) for evaluation and 80% (120 samples) for training.
- **Stratified Split:** stratify=y ensures that each of the 3 target flower species (setosa, versicolor, virginica) is equally represented in both train and test splits.

2. Model Training with Support Vector Machine (LinearSVC)

```
model = LinearSVC(max_iter=10000, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

- **Linear Support Vector Classifier (LinearSVC):** SVM seeks to find optimal hyperplanes in a multi-dimensional space that maximize the margin between classes.
- **Multiclass Strategy:** For datasets with more than two classes, LinearSVC automatically implements a **One-vs-Rest (OvR)** approach, fitting 3 separate linear boundaries (one per flower species).
- **max_iter=10000:** Increases the maximum iterations allowed for optimization to ensure the algorithm converges properly without raising a convergence warning.

3. Output Predictions Inspection

```
predictions_summary = pd.DataFrame({
    "Sample Index": X_test.index[5:10],
    "Actual Class": [iris.target_names[i] for i in y_test[5:10]],
    "Predicted Class": [iris.target_names[i] for i in y_pred[5:10]]
})
```

```
print(tabulate(predictions_summary, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

- Extracts samples at slice 5:10 from the test set and maps class indices back to actual species names (setosa, versicolor, virginica) to display a formatted comparison table.

4. Model Evaluation & Metrics

```
# Confusion Matrix
cm_dataframe = pd.DataFrame(
    confusion_matrix(y_test, y_pred),
    index=iris.target_names,
    columns=iris.target_names
)

# Accuracy & Metrics Report
accuracy = accuracy_score(y_test, y_pred)
report_dict = classification_report(y_test, y_pred,
target_names=iris.target_names, output_dict=True)
report_dataframe = pd.DataFrame(report_dict).transpose()
```

Evaluates performance across the 30 test instances:

- **Confusion Matrix:** Shows true vs. predicted species to visually highlight any instances misclassified across class boundaries.
- **Accuracy Score:** Measures the overall proportion of correctly classified instances ($\frac{\text{Correct}}{\text{Total}}$).
- **Classification Report:** Displays detailed metrics per class:
  - **Precision:** Accuracy of positive class predictions.
  - **Recall:** Fraction of actual class instances correctly identified by the model.
  - **F1-Score:** Harmonic mean balancing Precision and Recall.

Lab7

This script demonstrates **Unsupervised Learning** using the **K-Means Clustering** algorithm to segment users based on their movie genre preferences (Action, SciFi, Romance, Comedy). Here is the step-by-step breakdown of how the code works:

1. Dataset Creation

```
dataset_map = {
    'Action_Rating': [5, 4, 1, 1, 5, 2, 1, 5, 2, 4, 1, 5, 2, 4, 1],
    'SciFi_Rating': [5, 5, 2, 1, 4, 1, 2, 5, 1, 4, 2, 5, 1, 5, 1],
    'Romance_Rating': [1, 1, 5, 4, 2, 4, 5, 1, 5, 2, 4, 1, 4, 1, 5],
    'Comedy_Rating': [2, 2, 4, 5, 1, 5, 4, 2, 4, 1, 5, 2, 5, 2, 4]
}
ratings_df = pd.DataFrame(dataset_map)
```

- **Movie Ratings:** Creates a dataset of 15 users, where each row represents user ratings (on a scale of 1–5) across four genres. Notice how patterns exist in the raw numbers: some users prefer Action/SciFi while giving low ratings to Romance/Comedy, and vice versa.

2. K-Means Model Fitting

```
model_kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
model_kmeans.fit(ratings_df)
```

- **n_clusters=3:** Instructs the algorithm to group the 15 users into 3 distinct user persona clusters.
- **Algorithm Mechanism:** K-Means randomly initializes 3 centroids, assigns each user to the nearest centroid using Euclidean distance, recalculates the cluster centers based on mean ratings, and iterates until convergence.
- **n_init=10:** Runs the algorithm 10 times with different centroid seeds to select the output with the lowest inertia (sum of squared distances to closest centroid).

3. Cluster Centroids (User Archetypes)

```
centroids_df = pd.DataFrame(
    model_kmeans.cluster_centers_,
    columns=ratings_df.columns,
    index=[f"Cluster {i}" for i in range(3)]
)
print(tabulate(centroids_df.round(2), headers='keys',
tablefmt='fancy_grid', showindex=True))
```

- **cluster_centers_:** Represents the average ratings of all users within each cluster across the 4 genres.
- **Interpretation:** Examining these centroids reveals distinct audience personas (e.g., Cluster 0 = Action/SciFi fans, Cluster 1 = Romance/Comedy lovers, Cluster 2 = Casual/Mixed viewers).

4. User Cluster Assignment

```
labels_df = pd.DataFrame({
    'User': [f"U{i+1}" for i in range(len(model_kmeans.labels_))],
    'Cluster': [f"Cluster {c}" for c in model_kmeans.labels_]
})
print(tabulate(labels_df, headers='keys', tablefmt='fancy_grid',
showindex=False))
```

- **labels_:** Retrieves the cluster index (0, 1, or 2) assigned to each of the original 15 training users (U1 to U15) and formats them into a clean tabular layout.

5. Assigning New Unseen User

```
unseen_users = pd.DataFrame({
    'Action_Rating': [5, 1, 3, 2],
    'SciFi_Rating': [4, 1, 2, 2],
    'Romance_Rating': [1, 5, 4, 2],
    'Comedy_Rating': [2, 4, 5, 5]
})
target_predictions = model_kmeans.predict(unseen_users)
```

- **Generalization (.predict()):** Passes 4 new users (New User 1 through New User 4) through the trained model.
- **Nearest Centroid Assignment:** Measures the Euclidean distance between each new user's rating vector and the 3 learned cluster centroids, assigning the user to the closest cluster profile.

Lab 8

This script demonstrates scalable unsupervised clustering using **Mini-Batch K-Means** on synthetic 2D cluster data generated with Scikit-Learn.
Here is the step-by-step breakdown:

1. Synthetic Data Generation

```
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0,
random_state=42)
feature_names = ['Feature_1', 'Feature_2']
blobs_df = pd.DataFrame(X, columns=feature_names)
```

- **make_blobs:** Generates 300 isotropic Gaussian blobs (clusters) in 2D space grouped around 3 distinct centers.
- **DataFrame:** Wraps the coordinates into a Pandas DataFrame with columns Feature_1 and Feature_2.

2. Mini-Batch K-Means Model Fitting

```
model_mbkm = MiniBatchKMeans(n_clusters=3, batch_size=30, random_state=42,
n_init=10)
model_mbkm.fit(blobs_df)
```

- **Mini-Batch K-Means vs. Standard K-Means:** Standard K-Means evaluates the entire dataset during every iteration. Mini-Batch K-Means updates centroid positions using small, random subsets (**mini-batches**) of size 30 (batch_size=30). This drastically cuts down computation time and memory usage for large datasets while producing very similar results.
- **n_clusters=3:** Target number of clusters to discover.

3. Centroids & Prediction on Unseen Points

```
centroids_df = pd.DataFrame(model_mbkm.cluster_centers_,
columns=feature_names)

unseen_points = np.array([[-2.5, 9.0], [4.0, 2.0], [-6.0, -7.0], [2.0,
1.5]])
new_users_df = pd.DataFrame(unseen_points, columns=feature_names)
new_users_df['Assigned Cluster'] = model_mbkm.predict(new_users_df)
```

- **Cluster Centroids:** Extracts the 2D coordinates of the 3 learned cluster centers.

- **Inference (.predict()):** Assigns 4 new, unseen 2D data points to their nearest cluster center based on minimum Euclidean distance.

4. Cluster Visualization & File Saving

```
fig, ax = plt.subplots(figsize=(6, 4))
colors = ['#1f77b4', '#ff777e', '#2ca02c']

# Plot cluster points
for i in range(3):
    cluster_data = blobs_df[model_mbkm.labels_ == i]
    ax.scatter(cluster_data['Feature_1'], cluster_data['Feature_2'],
c=colors[i], label=f'Cluster {i}', alpha=0.6, edgecolors='w', s=30)

# Plot centroids
ax.scatter(model_mbkm.cluster_centers_[:, 0],
model_mbkm.cluster_centers_[:, 1], c='black', marker='X', s=150,
label='Centroids')

plt.savefig("minibatch_kmeans_clusters.png", dpi=300, bbox_inches='tight')
plt.show()
```

- **Scatter Plot:** Plots all 300 data points color-coded by their assigned cluster label.
- **Centroid Markers:** Overlays the learned centroids as prominent black **X** markers ($s=150$).
- **Export:** Saves the plot as minibatch_kmeans_clusters.png at 300 DPI.

Lab 9

This script attempts to demonstrate **K-Medoids Clustering**, though it uses Scikit-Learn's KMeans class as a placeholder/fallback implementation.
Here is the step-by-step breakdown of how the code works and the important distinction between K-Means and K-Medoids:

1. Synthetic Dataset Generation

```
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0,
random_state=42)
feature_names = ['Feature_1', 'Feature_2']
blobs_df = pd.DataFrame(X, columns=feature_names)
```

- **make_blobs:** Generates 300 synthetic 2D data points clustered around 3 center points.
- **DataFrame:** Structures the coordinates into a Pandas DataFrame with features Feature_1 and Feature_2.

2. Clustering Model Fitting & Conceptual Difference

```
model_km = KMeans(n_clusters=3, random_state=42, n_init=10)
model_km.fit(blobs_df)
centroids_df = pd.DataFrame(model_km.cluster_centers_,
columns=feature_names)
```

- **Fallback Implementation:** Standard Scikit-Learn does not have a native KMedoids class (which is usually imported from scikit-learn-extra), so the code substitutes KMeans to estimate cluster centers.
- **K-Means vs. K-Medoids Difference:**
  - **K-Means (Centroids):** Calculates the continuous mean position of points in a cluster. The centroid is often a synthetic coordinate that does not exist in the original dataset.
  - **K-Medoids (Medoids):** Requires the center of a cluster to be an **actual data point** from the dataset (minimizing absolute pairwise distances instead of squared Euclidean distances). It is much less sensitive to extreme outliers than K-Means.

3. Classifying Unseen Data

```
unseen_points = np.array([[-2.5, 9.0], [4.0, 2.0], [-6.0, -7.0], [2.0,
1.5]])
new_users_df = pd.DataFrame(unseen_points, columns=feature_names)
new_users_df['Assigned Cluster'] = model_km.predict(new_users_df)
```

- Accepts 4 unseen test points, calculates their distance to the cluster centers via .predict(), and assigns each point to its nearest cluster index (0, 1, or 2).

4. Plotting & Saving Results

```
fig, ax = plt.subplots(figsize=(6, 4))
colors = ['#1f77b4', '#ff777e', '#2ca02c']

for i in range(3):
    cluster_data = blobs_df[model_km.labels_ == i]
    ax.scatter(cluster_data['Feature_1'], cluster_data['Feature_2'],
               c=colors[i], label=f'Cluster {i}', alpha=0.6,
edgecolors='w', s=30)

ax.scatter(model_km.cluster_centers_[:, 0], model_km.cluster_centers_[:,
1],
           c='black', marker='X', s=150, label='Medoids')

plt.savefig("kmedoids_clusters.png", dpi=300, bbox_inches='tight')
plt.show()
```

- Plots all 300 sample points color-coded by cluster membership.
- Highlights the cluster centers (labeled as "Medoids" in the legend) using black **X** markers ($s=150$).
- Saves the resulting figure as kmedoids_clusters.png at 300 DPI.

Lab 10

This script demonstrates **Agglomerative Hierarchical Clustering** on a small dataset of animal physical traits using SciPy. It computes and compares dendrograms across five different linkage methods.
Here is the step-by-step breakdown:

1. Feature Matrix & Indexing

```
df = pd.DataFrame(animal_features, index=animal_names)
```

- **Dataset:** Structures binary traits (Hair, Feathers, Eggs, Milk, etc.) across 10 animals (Lion, Dolphin, Eagle, etc.) into a DataFrame.
- **Agglomerative approach:** Starts with every animal in its own separate cluster and iteratively merges the most similar pair of clusters until only one cluster remains.

2. Hierarchical Linkage Calculation (linkage())

```
linkage_methods = ['single', 'complete', 'average', 'centroid', 'ward']
...
Z = linkage(df, method=method)
```

The script runs scipy.cluster.hierarchy.linkage on the dataset across 5 different distance algorithms to construct the cluster hierarchy:

- **Single Linkage:** Measures proximity using the distance between the two **closest** points in separate clusters (can cause "chaining").
- **Complete Linkage:** Measures proximity using the distance between the two **farthest** points in separate clusters.
- **Average Linkage:** Uses the average distance between all pairs of objects across two clusters.
- **Centroid Linkage:** Calculates the distance between the **centers (geometric centroids)** of two clusters.
- **Ward's Linkage:** Minimizes the total **within-cluster variance** (sum of squared deviations from the cluster mean) when merging clusters.

3. Subplot Generation & Dendrograms (dendrogram())

```
plt.subplot(3, 2, i + 1)
dendrogram(Z, labels=df.index, leaf_rotation=45)
```

- **Tree Visualization:** Generates a **dendrogram** for each linkage method.
- **Horizontal Lines:** The height of the horizontal bars on a dendrogram indicates the

dissimilarity (distance) at which two clusters were merged together.

4. Plot Styling & Saving

```
plt.tight_layout()
plt.suptitle('Agglomerative Hierarchical Clustering - Different Linkage
Methods', y=1.02, fontsize=11)
plt.savefig('animal_dendrograms_modified.png', dpi=200,
bbox_inches='tight')
plt.show()
```

- Arranges all 5 dendrogram subplots into a 3×2 grid layout.
- Saves the combined figure as animal_dendrograms_modified.png at 200 DPI.
- Prints a terminal summary listing all five evaluated linkage methods.

lab 11

This script demonstrates **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**, a density-based clustering algorithm capable of discovering clusters of arbitrary shape and automatically detecting noise/outliers.
Here is the step-by-step breakdown of how the code works:

1. 2D Synthetic Dataset Setup & Initial Plot

```
data = np.array([
    [2.0, 2.0], [2.2, 2.4], [1.8, 2.2], [2.5, 1.8], [2.1, 2.1], # Group 1
    [8.0, 8.0], [8.5, 8.2], [7.8, 8.5], [8.2, 7.7], [8.1, 8.1], # Group 2
    [2.0, 8.0], [2.3, 8.2], [1.9, 7.8], [2.1, 8.4], [2.2, 7.9], # Group 3
    [14.0, 14.0], [14.5, 13.8], [13.7, 14.2], [14.2, 14.5], [13.9, 13.9], #
Group 4
    [0.0, 15.0], [7.0, 0.0], [15.0, 2.0] # Isolated points (Outliers)
```

- **Dataset Structure:** Contains 23 total 2D points. 20 of these points form 4 tight, dense groups of 5 points each. The remaining 3 points ([0, 15], [7, 0], and [15, 2]) are isolated and far away from any group.
- **Initial Plot:** Plots all raw points in purple and saves the scatter plot as dbscan_initial_plot_modified.png.

2. DBSCAN Fitting & Hyperparameters

```
dbscan = DBSCAN(eps=1.5, min_samples=4)
labels = dbscan.fit_predict(data)
```

DBSCAN relies on two key hyperparameters:

- **eps ($\epsilon = 1.5$):** The maximum distance neighborhood radius around a point.
- **min_samples (4):** The minimum number of points required within a point's ϵ-neighborhood for it to be considered a **Core Point**.

How DBSCAN assigns labels:

1. **Core Points:** Points with at least 4 neighbors within a distance of 1.5.
2. **Border Points:** Points within ϵ-distance of a Core Point, but having fewer than min_samples neighbors themselves.
3. **Noise Points (-1):** Points that are neither Core nor Border points.

3. Cluster Visualization & Noise Separation

```
if k == -1:
    plt.scatter(data[class_member_mask, 0], data[class_member_mask, 1],
                s=100, marker='x', c='black', label='Noise')
```

```
else:
    plt.scatter(data[class_member_mask, 0], data[class_member_mask, 1],
                s=100, c=colors[i % len(colors)], label=f'Cluster {k}')
```

- **Cluster Points ($k \ge 0$):** Plotted as filled circles with distinct colors per cluster.
- **Noise Points ($k = -1$):** Plotted using black **X** markers to highlight outliers.
- **Export:** Saves the final clustered plot as dbscan_clusters_modified.png.

4. Cluster Summary Output

```
num_clusters = len(set(labels)) - (1 if -1 in labels else 0)
num_noise = list(labels).count(-1)
```

- **Calculates performance metrics:**
  - Subtracts 1 from the label count if -1 exists so that noise is not counted as a valid cluster.
  - Counts the total number of isolated points labeled as noise (-1).
- **Expected Output:** Identifies **4 clusters** and **3 noise points**.